



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Lenguajes y Gramáticas Formales

Equipo: **VISIONLY**

Mantenlo simple!

Profesor:

Felix Marquez

Integrantes:

Ana Santamaria V-21251440

Carlos Martínez V-27955202

Genesis Moya V-31370339

Naleska Carrera V-31521506

Ciudad Guayana, junio 19 del 2026.

INTRODUCCIÓN

Los lenguajes formales constituyen uno de los fundamentos teóricos más importantes de la informática moderna, ya que proporcionan los mecanismos necesarios para definir, modelar y procesar información de manera precisa y estructurada. En el ámbito de los lenguajes de programación, estos conceptos permiten comprender cómo se construyen los compiladores e intérpretes encargados de traducir el código fuente a instrucciones ejecutables por una computadora.

El presente informe aborda los principales elementos relacionados con los lenguajes y gramáticas formales, comenzando con el estudio de la relación entre gramáticas y lenguajes, así como la clasificación establecida por la Jerarquía de Chomsky. Posteriormente, se analizan los conceptos de derivación y modelado mediante gramáticas libres de contexto, destacando su aplicación en la representación de estructuras complejas.

Asimismo, se estudian las principales patologías que pueden presentarse en las gramáticas, tales como la ambigüedad, la recursividad por la izquierda y la necesidad de factorización, examinando los métodos utilizados para optimizar y garantizar un procesamiento sintáctico eficiente. Finalmente, se exploran las expresiones regulares y los autómatas finitos determinísticos como herramientas fundamentales para el reconocimiento de patrones y la validación de lenguajes formales, aplicándolos a un caso práctico basado en la notación PGN utilizada en el ajedrez.

A través de esta investigación se busca fortalecer la comprensión de los principios teóricos que sustentan el diseño de compiladores y sistemas de procesamiento de lenguajes, desarrollando habilidades de análisis formal y pensamiento computacional necesarias para la formación de un ingeniero en informática.

2.1.- Fundamentos y Jerarquía (Teoría Aplicada).

2.2.1.- Relación Gramática-Lenguaje:

Las gramáticas formales constituyen el núcleo estratégico sobre el cual se erige el procesamiento de lenguajes, actuando como un puente indispensable entre la lógica matemática pura y la capacidad operativa de los sistemas de computación. En el ámbito de la informática teórica, la precisión en la definición de una gramática no es una mera formalidad académica; es el requisito *sine qua non* para garantizar la fiabilidad computacional. Sin un rigor estructural absoluto en la definición de las reglas que gobiernan un lenguaje, la traducción de una intención algorítmica en una instrucción de máquina carecería de la consistencia necesaria para el desarrollo de sistemas robustos y predecibles.

Desde una perspectiva formal, una gramática se define matemáticamente como una cuaterna $G = (V, \Sigma, S, P)$, donde cada componente cumple un rol estructural específico y diferenciado:

- **V (Símbolos no terminales):** Conjunto finito de variables sintácticas que definen la jerarquía y estructura del lenguaje.
- **Σ (Símbolos terminales):** Alfabeto básico que compone las cadenas finales del lenguaje. Es imperativo notar la propiedad de exclusión mutua: $V \cap \Sigma = \emptyset$, lo que garantiza la ausencia de ambigüedad entre los elementos de estructura y los de contenido.
- **S (Símbolo inicial):** El axioma o variable perteneciente a V ($S \in V$) que marca el punto de partida de todo proceso de generación.
- **P (Producciones):** Conjunto finito de reglas de reescritura de la forma $\alpha \rightarrow \beta$. De manera general, una producción se define como una relación entre cadenas, donde la parte izquierda debe contener al menos un símbolo no terminal: $P \subseteq (V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^*$.

El dinamismo de esta estructura se manifiesta a través del mecanismo de derivación, un sistema de transición que opera sobre el espacio de configuraciones definido por $(V \cup \Sigma)^*$. Este proceso se formaliza mediante la derivación en un paso ($\Rightarrow$), donde una cadena se transforma en otra aplicando una regla de P . Para describir procesos completos, empleamos la clausura reflexiva y transitiva ($\Rightarrow^*$), que representa una derivación en cero o más pasos, y la clausura transitiva ($\Rightarrow^+$), que implica al menos un paso de aplicación. Así, el lenguaje generado se define rigurosamente como el conjunto de cadenas terminales derivables desde el axioma: $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$.

La interpretación semántica de un lenguaje depende intrínsecamente de su estructura sintáctica, visualizada mediante el árbol de derivación. La efectividad de este proceso puede verse comprometida por la ambigüedad, fenómeno que ocurre cuando una cadena posee más de un árbol de derivación (o múltiples derivaciones canónicas, ya sean por la izquierda o por la derecha). La capacidad de una gramática para generar una estructura jerárquica unívoca es lo que permite que un compilador asigne un significado único a cada instrucción. Esta libertad de generación, no obstante, requiere un orden sistemático, lo que nos conduce a la necesidad de imponer restricciones estructurales según la Jerarquía de Chomsky.

2.2.2.- Jerarquía de Chomsky:

En 1956, Noam Chomsky transformó el estudio de la lingüística matemática al proponer una clasificación taxonómica que organiza las gramáticas según la complejidad de sus reglas de producción. Esta jerarquía establece un modelo de inclusión matemática donde cada nivel superior es un subconjunto propio del anterior, imponiendo restricciones adicionales que dictan tanto el poder expresivo del lenguaje como la potencia de la máquina necesaria para su reconocimiento.

Los Cuatro Niveles de la Jerarquía

1. Tipo 0: Gramáticas Sin Restricciones (Recursivamente Enumerables)

- **Restricción:** Son las más generales, sólo requieren que en el lado izquierdo de la regla exista al menos un símbolo no terminal.
- **Forma matemática:** $\alpha A \beta \rightarrow \delta$, donde $A \in V$ y $\alpha, \beta, \delta \in (V \cup \Sigma)^*$.
- **Poder de cómputo:** Reconocidas por la Máquina de Turing, el modelo universal de computación.

2. Tipo 1: Gramáticas Sensibles al Contexto

- **Restricción:** Un símbolo solo se sustituye dentro de un contexto específico. Son fundamentalmente no compresoras, lo que implica que la longitud de la cadena resultante debe ser mayor o igual a la original: $|\alpha| \leq |\beta|$.
- **Forma matemática:** $\alpha A \beta \rightarrow \alpha \gamma \beta$, donde $\gamma \in (V \cup \Sigma)^+$ (no puede ser vacía).
- **Poder de cómputo:** Reconocidas por autómatas linealmente acotados.

3. Tipo 2: Gramáticas Libres de Contexto (Independientes del contexto)

- **Restricción:** El reemplazo de un símbolo no terminal es absoluto y no depende de los símbolos adyacentes. El lado izquierdo debe ser estrictamente un único no terminal.
- **Forma matemática:** $A \rightarrow \gamma$, donde $A \in V$ y $\gamma \in (V \cup \Sigma)^*$.
- **Poder de cómputo:** Base de la sintaxis de los lenguajes de programación; reconocidas por autómatas con pila.

4. Tipo 3: Gramáticas Regulares

- **Restricción:** El nivel más restrictivo. Las producciones son lineales (por la derecha o por la izquierda), permitiendo como máximo un no terminal en posiciones específicas.
- **Forma matemática:** $A \rightarrow aB$ o $A \rightarrow a$ (lineales por la derecha).
- **Poder de cómputo:** Expresables mediante expresiones regulares y aceptadas por autómatas finitos.

Síntesis Comparativa de la Jerarquía

Tipo	Denominación	Restricción de Producción	Propiedad Matemática	Autómata
0	Sin restricciones	$\alpha A \beta \rightarrow \delta$	Ninguna	Máquina de Turing
1	Sensible al contexto	$\alpha A \beta \rightarrow \alpha \gamma \beta$	No compresora ($ \alpha \leq \beta $)	Autómata linealmente acotado
2	Libre de contexto	$A \rightarrow \gamma$	Lado izq. único no terminal	Autómata con pila
3	Regular	$A \rightarrow aB \mid a$	Linealidad estricta	Autómata finito

La relación entre la restricción de la regla y la complejidad computacional es inversamente proporcional: a mayores restricciones, mayor es la eficiencia y la decidibilidad del análisis. Los lenguajes de programación modernos se centran en el Tipo 2 porque ofrecen el equilibrio óptimo: suficiente potencia para estructuras recursivas y una complejidad de procesamiento gestionable por compiladores eficientes. Esta estructuración teórica se materializa en la práctica mediante la notación Backus-Naur (BNF).

Ejemplos prácticos representados en notación BNF (Backus-Naur Form):

La forma Backus-Naur (BNF) es el estándar académico para describir la sintaxis de lenguajes formales, destacando por su transparencia al representar la recursividad. Es fundamental notar que, aunque la notación BNF fue diseñada originalmente para gramáticas de Tipo 2, se emplea a continuación como un lenguaje de marcado estructural para ilustrar los diversos niveles de la jerarquía de Chomsky.

A continuación, se presentan cuatro modelos bajo la sintaxis `<variable> ::= expresión`:

1. Tipo 3 (Regular):

Se emplean para el reconocimiento de patrones de texto lineales en el código fuente, como la definición de un identificador de variable elemental que debe comenzar con una letra y completarse de forma lineal con letras o dígitos.

- $\langle \text{Identificador} \rangle ::= \langle \text{Letra} \rangle$
- $\langle \text{Identificador} \rangle ::= \langle \text{Letra} \rangle \langle \text{Resto} \rangle$
- $\langle \text{Resto} \rangle ::= \langle \text{Letra} \rangle$
- $\langle \text{Resto} \rangle ::= \langle \text{Digito} \rangle$
- $\langle \text{Resto} \rangle ::= \langle \text{Letra} \rangle \langle \text{Resto} \rangle$
- $\langle \text{Resto} \rangle ::= \langle \text{Digito} \rangle \langle \text{Resto} \rangle$

2. Tipo 2 (Libre de Contexto):

Permiten modelar la recursividad y el anidamiento de los bloques en los lenguajes de programación (por ejemplo, comprobar que las llaves o los paréntesis de apertura y cierre dentro de funciones complejas estén balanceados).

- $\langle S \rangle ::= ()$
- $\langle S \rangle ::= (\langle S \rangle)$
- $\langle S \rangle ::= \langle S \rangle \langle S \rangle$

3. Tipo 1 (Sensible al Contexto):

En este nivel, la BNF se adapta para mostrar la interdependencia de símbolos. El lenguaje requiere que el número de 'a', 'b' y 'c' sea idéntico, lo cual no es posible sin reglas que consideren el entorno de los símbolos.

- $\langle S \rangle ::= abc$
- $\langle S \rangle ::= a \langle S \rangle \langle B \rangle c$
- $\langle B \rangle b ::= bb$ (Regla de contexto: B se convierte en 'b' ante otra 'b')
- $\langle B \rangle c ::= bc$ (Regla de contexto: B se posiciona ante 'c')

4. Tipo 0 (Sin restricciones):

No se utilizan para estructurar el código sintáctico comercial debido a que sus transformaciones arbitrarias pueden generar bucles infinitos de sustitución (problemas lógicos indecidibles). Su valor radica en establecer la frontera teórica de la computación profunda.

- $\langle A \rangle \langle B \rangle ::= \langle C \rangle$
- $\langle C \rangle \langle D \rangle ::= \langle D \rangle \langle A \rangle$
- $\langle S \rangle ::= \langle A \rangle \langle B \rangle \langle D \rangle$

La robustez de estos modelos varía significativamente. Mientras que el Tipo 3 es extremadamente eficiente pero incapaz de manejar estructuras anidadas complejas, el Tipo 0 ofrece una potencia computacional total (Turing-completo) a costa de una complejidad que puede derivar en problemas indecidibles. Esta graduación justifica por qué la ingeniería de lenguajes se apoya predominantemente en el equilibrio del Tipo 2 para la construcción de compiladores e intérpretes modernos.

2.2.- Derivación y Modelado (Caso Práctico: Genoma).

2.2.1.- Gramática del para dibujar: Basado en una Gramática Libre de Contexto (GLC) que modele herramientas de dibujo (alfabeto $\Sigma = \{a, c, g, t\}$).

Una Gramática Libre de Contexto (GLC) es un formalismo matemático utilizado para definir y generar la estructura sintáctica de lenguajes, tanto naturales como formales (lenguajes de programación, protocolos de datos, etc.). Su característica distintiva y por la cual recibe el adjetivo "libre de contexto" radica en que sus reglas de producción o derivación siempre tienen un único símbolo "no terminal" en el lado izquierdo. Esto significa que la regla para expandir ese símbolo se puede aplicar de manera absoluta, sin importar qué otros símbolos lo rodeen (su contexto) en la cadena que se está evaluando.

En términos prácticos, una GLC actúa como un conjunto estricto de reglas de sustitución. Comienza con un axioma o símbolo inicial y, mediante reemplazos sucesivos basados en estas reglas, genera secuencias de símbolos "terminales" (las unidades indivisibles del lenguaje, como palabras en un texto, o instrucciones de dibujo en nuestro caso genómico). Esta capacidad estructurada permite modelar desde la sintaxis de bucles y condicionales en compiladores informáticos, hasta las complejas jerarquías de ramificación en organismos biológicos, como se evidencia en los Sistemas de Lindenmayer o L-System

1. Definición Formal del Modelo

Definimos la Gramática Libre de Contexto como la tupla $G=(V,\Sigma,P,S)$:

- **Alfabeto Terminal (Σ):** $\{a,c,g,t\}$
- **Símbolos No Terminales (V):** $\{S,Q,L,C,R,A,B,P\}$
- **Símbolo Inicial:** S
- **Reglas de Producción (P):** Definidas en la sección 3.

Semántica Geométrica (El Motor de Trazado)

Para que los 4 símbolos formen figuras ortogonales y diagonales, configuraremos el motor asumiendo que los giros son de **45 grados** ($\theta=45^\circ$) y la distancia de avance es constante (D).

- **a (Avanzar):** Mueve el cursor hacia adelante trazando una línea.
- **c (Cruzar derecha):** Rota el cursor $+45^\circ$ en sentido horario.
- **g (Girar izquierda):** Rota el cursor -45° en sentido antihorario.
- **t (Tronco/Teletransportar):** Retrocede el cursor la distancia D sin cambiar el ángulo de orientación. Esencial para bifurcaciones y volver a los vértices.

2. Reglas de Producción Estructuradas (P)

Para mantener la gramática modular y escalable (como un buen diseño de backend), separamos las lógicas en distintos no terminales:

```

$$S \rightarrow Q \mid R \mid A \mid C \mid P$$

$$L \rightarrow a c c \quad (\text{Avanza y gira } 90^\circ \text{ para formar esquinas ortogonales})$$

$$Q \rightarrow L L L a \quad (\text{Lógica base de un cuadrado de 4 lados})$$

$$R \rightarrow a c a t g g a \quad (\text{Rama bifurcada en "Y"})$$

$$B \rightarrow a c B t g g B \mid a \quad (\text{Bifurcación recursiva para árboles densos})$$

$$A \rightarrow B \quad (\text{Regla de inicio para el Árbol})$$

$$C \rightarrow Q t c a g Q \quad (\text{Proyección isométrica base de un cubo})$$

```

3. Las 5 Derivaciones Propuestas

Las derivaciones presentadas a continuación son **derivaciones por la izquierda** (Leftmost Derivation), expandiendo siempre el no terminal más a la izquierda primero.

a) Derivación 1: Segmento Base (Tallo Simple)

El caso más elemental para inicializar una ruta.

$$S \Rightarrow P \Rightarrow a$$

Resultado: Una línea recta simple. La cadena genómica final es *a*.

b) Derivación 2: El Cuadrado (Q)

Requiere giros de 90° . Como nuestro motor gira 45° , usamos *cc*.

$$\begin{aligned} S &\Rightarrow Q \\ &\Rightarrow L L L a \\ &\Rightarrow (a c c) L L a \\ &\Rightarrow a c c (a c c) L a \\ &\Rightarrow a c c a c c (a c c) a \\ &\Rightarrow a c c a c c a c c a \end{aligned}$$

Resultado: La máquina avanza, gira 90° , avanza, gira 90°
... cerrando un cuadrado. Genoma: a c c a c c a c c a.

c) Derivación 3: Rama Simple (R)

Una línea que se divide en dos (una hoja a la derecha y otra a la izquierda), volviendo al punto de bifurcación mediante la instrucción t .

$S \Rightarrow R$
 $\Rightarrow a c a t g g a$

Explicación del Genoma: Avanza (a), gira 45° a la derecha (c), traza la hoja derecha (a), retrocede a la bifurcación (t), gira 90° a la izquierda para apuntar a la otra dirección (gg), traza la hoja izquierda (a).

d) Derivación 4: Árbol con Varias Ramas y Hojas (A)

Utilizamos recursividad para inyectar complejidad fractal, simulando el crecimiento de un organismo. Limitaremos la profundidad a 1 nivel de recursión para el ejemplo manual.

$S \Rightarrow A$
 $\Rightarrow B$
 $\Rightarrow a c B t g g B$ (Expansion recursiva del tronco principal)
 $\Rightarrow a c (a) t g g B$ (Hoja derecha terminal)
 $\Rightarrow a c a t g g (a c B t g g B)$ (Expansion de la rama izquierda)
 $\Rightarrow a c a t g g a c (a) t g g B$
 $\Rightarrow a c a t g g a c a t g g (a)$

Resultado: Un árbol asimétrico con sub-ramas. La recursión permite que el genoma escale a estructuras botánicas hiperrealistas.

e) Derivación 5: El Cubo / Proyección 2D (C)

Para trazar la ilusión de un cubo (proyección oblicua), se dibuja la cara frontal, se retrocede con un ángulo a una esquina trasera, y se dibuja la cara posterior.

$S \Rightarrow C$

$\Rightarrow Q t c a g Q$

$\Rightarrow (L L L a) t c a g Q$

$\Rightarrow (a c c a c c a c c a) t c a g Q$

$\Rightarrow a c c a c c a c c a t c a g (L L L a)$

$\Rightarrow a c c a c c a c c a t c a g (a c c a c c a c c a)$

Resultado: La lógica dibuja el cuadrado frontal, utiliza t para volver sobre su trazo y posicionarse geoméricamente para trazar un enlace diagonal (ca), y finalmente vuelve a la orientación original (g) para proyectar el segundo cuadrado.

A continuación, he diseñado un simulador del autómata donde puedes probar los genomas derivados. Al ingresar secuencias del alfabeto Σ , el widget ejecutará la lógica en tiempo real para renderizar las figuras y validar tu asignación.

2.3.- Higiene y Optimización de Gramáticas.

2.3.1.- Patologías de las Gramáticas: Las gramáticas mal diseñadas causan errores en los compiladores. Ejemplifique con 3 casos prácticos diferentes:

a) Gramática Ambigua

Una gramática es ambigua si puede generar más de un árbol de derivación sintáctica para una misma cadena de entrada. Esto es problemático porque el compilador no sabría qué interpretación de la estructura jerárquica es la correcta.

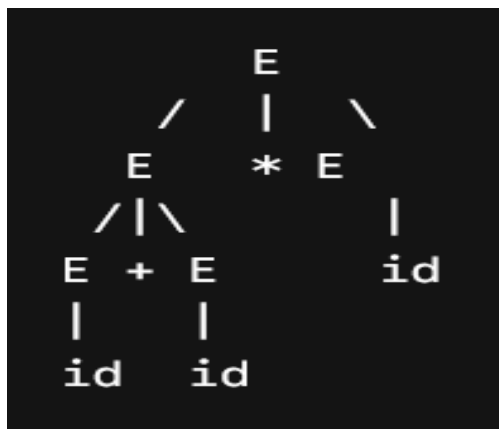
Caso práctico: Evaluemos la gramática clásica de expresiones aritméticas.

$E \rightarrow E + E \mid E * E \mid id$

Para la cadena: $id + id * id$

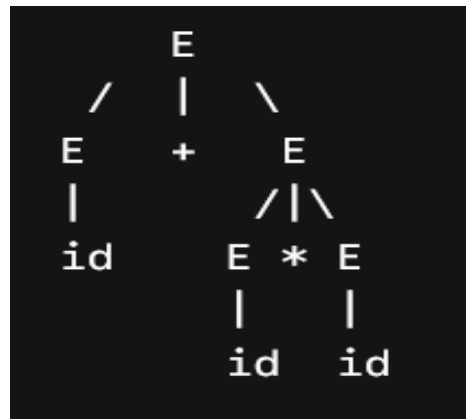
Demostración de Ambigüedad (Dos árboles distintos):

Árbol 1: Precedencia incorrecta (Suma antes que multiplicación)



Interpretación: Se evalúa primero $(id + id)$ y luego se multiplica.

Árbol 2: Precedencia correcta (Multiplicación antes que suma)



Interpretación: Se evalúa primero `(id*id)` y luego se suma.

Solución en Python:

```
1 class ParserDescendenteRecurso:
2     def __init__(self, tokens):
3         self.tokens = tokens
4         self.pos = 0
5
6     def obtener_token_actual(self):
7         """Devuelve el token en la posición actual de lectura
8         ."""
9         if self.pos < len(self.tokens):
10             return self.tokens[self.pos]
11         return None
12
13     def match(self, token_esperado):
14         """Avanza si el token actual coincide con el esperado
15         ."""
16         if self.obtener_token_actual() == token_esperado:
17             self.pos += 1
18             return True
19         return False
20
21     def parse_E(self):
22         """
23         Regla Expresión (E -> T + E | T)
24         Maneja la suma (Menor precedencia).
25         """
26         nodo_izq = self.parse_T()
```

```

        jerárquica
27 ▾     if self.obtener_token_actual() == '+':
28         op = self.obtener_token_actual()
29         self.pos += 1 # Consumir el '+'
30         nodo_der = self.parse_E()
31         return f"[{nodo_izq} {op} {nodo_der}]"
32
33     return nodo_izq
34
35 ▾ def parse_T(self):
36     """
37     Regla Término (T -> F * T | F)
38     Maneja la multiplicación (Mayor precedencia).
39     """
40     nodo_izq = self.parse_F()
41
42     # Si encontramos un signo '*', se agrupa primero aquí
43 ▾     if self.obtener_token_actual() == '*':
44         op = self.obtener_token_actual()
45         self.pos += 1 # Consumir el '*'
46         nodo_der = self.parse_T()
47         return f"[{nodo_izq} {op} {nodo_der}]"
48
49     return nodo_izq
50
51 ▾ def parse_F(self):
52     """
53     Regla Factor (F -> id)

```

```

54         Maneja los elementos terminales (identificadores o
           variables).
55         """
56         token = self.obtener_token_actual()
57         if token == 'id':
58             self.pos += 1 # Consumir el 'id'
59             return "id"
60
61         raise SyntaxError(f"Error Sintáctico: Se esperaba 'id',
           se obtuvo '{token}'")
62
63     # =====
64     # PRUEBA DE RESOLUCIÓN DE AMBIGÜEDAD
65     # =====
66     if __name__ == "__main__":
67         # Cadena de entrada en forma de tokens: "id + id * id"
68         tokens_entrada = ['id', '+', 'id', '*', 'id']
69
70         print("Cadena de entrada:", " ".join(tokens_entrada))
71
72         # Inicializamos el parser con los tokens
73         parser = ParserDescendenteRecursoivo(tokens_entrada)
74
75         # Comenzamos el análisis desde el axioma inicial (E)
76         arbol_sintactico = parser.parse_E()
77
78         print("\n--- Árbol Sintáctico Único Generado ---")
79         print(arbol_sintactico)

```

Solución en Python:

```

Cadena de entrada: id + id * id

--- Árbol Sintáctico Único Generado ---
[id + [id * id]]

=== Code Execution Successful ===

```


b) Recursividad por la Izquierda

Una gramática es recursiva por la izquierda si tiene una producción de la forma $A \rightarrow A\alpha$, donde un no-terminal se llama a sí mismo como primer símbolo. Esto provoca bucles infinitos en los analizadores sintácticos descendentes (Top-Down).

Caso práctico y Algoritmo: Supongamos la regla para una lista de expresiones separadas por comas: $L \rightarrow L, id | id$

Algoritmo paso a paso para eliminarla:

- **Identificar la estructura base:** $A \rightarrow A\alpha | \beta$

Donde $A=L$

$\alpha=,id$ (lo que sigue a la recursión)

$\beta=id$ (el caso base)

- **Aplicar la transformación matemática:**

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' | \epsilon$ (donde ϵ es la cadena vacía)

- **Sustituir con nuestros valores:**

$L \rightarrow id L'$

$L' \rightarrow ,id L' | \epsilon$

Implementación del Algoritmo en Python: Podemos modelar este proceso de eliminación con un script simple que procese las reglas:

```

1  def eliminar_rekursividad_izq(no_terminal, producciones):
2      alphas = []
3      betas = []
4
5      # 1. Separar producciones recursivas (alpha) y no recursivas (beta)
6      for prod in producciones:
7          if prod.startswith(no_terminal):
8              alphas.append(prod[len(no_terminal):].strip())
9          else:
10             betas.append(prod)
11
12     # 2. Generar las nuevas reglas si hay recursividad
13     if not alphas:
14         return {no_terminal: producciones}
15
16     nuevo_nt = no_terminal + ""
17     nuevas_reglas = {
18         no_terminal: [f"{beta} {nuevo_nt}" for beta in betas],
19         nuevo_nt: [f"{alpha} {nuevo_nt}" for alpha in alphas] + ["ε"]
20     }
21
22     return nuevas_reglas
23
24 # Prueba del caso práctico
25 reglas_L = ["L , id", "id"]
26 resultado = eliminar_rekursividad_izq("L", reglas_L)

```

Salida en Python

```

L -> id L'
L' -> , id L' | ε

```

```
[Execution complete with exit code 0]
```

c) Factorización por la Izquierda

La factorización por la izquierda es una transformación gramatical útil para producir gramáticas aptas para el análisis sintáctico descendente (Top-Down, como los parsers LL(1)).

El problema surge cuando dos o más producciones de un mismo No-Terminal comienzan con la misma cadena de símbolos (el mismo prefijo). Al leer el primer token, el analizador predictivo no sabe qué producción elegir, lo que causa fallos o lo obliga a hacer un costoso "retroceso" (*backtracking*).

La Solución: Factorizar (extraer) el prefijo común en una sola regla y diferir la decisión introduciendo un nuevo No-Terminal para las terminaciones divergentes.

Caso Práctico (El problema del "Dangling Else")

Este es el caso clásico en el diseño de compiladores: la sentencia condicional `if`.

Gramática Original (Con problema de prefijo común):

- $S \rightarrow \text{if } E \text{ then } S$
- $S \rightarrow \text{if } E \text{ then } S \text{ else } S$
- $S \rightarrow \text{otro}$

Aquí, el analizador se confunde al ver el token `if`, ya que el prefijo común es $\alpha = \text{if } E \text{ then } S$.

Gramática Resultante Optimizada (Factorizada): Extraemos el prefijo común α y creamos un nuevo estado S' para lo restante ($\beta_1 = \epsilon$ y $\beta_2 = \text{else } S$):

- $S \rightarrow \text{if } E \text{ then } S S'$
- $S \rightarrow \text{otro}$
- $S' \rightarrow \text{else } S | \epsilon$

(Nota: ϵ representa la cadena vacía).

Implementación en Python:

```
1  from collections import defaultdict
2
3  def factorizar_izquierda(no_terminal, producciones):
4      """
5      Toma un No-Terminal y sus producciones, y aplica
6      factorización
7      por la izquierda agrupando automáticamente las reglas
8      conflictivas.
9      """
10     # 1. Agrupar las producciones por su primer token (palabra)
11     grupos = defaultdict(list)
12     for prod in producciones:
13         tokens = prod.split()
14         if tokens:
15             grupos[tokens[0]].append(tokens)
16
17     nuevas_reglas_base = []
18     nuevas_reglas_prima = []
19     nuevo_nt = f"{no_terminal}"
20
21     # 2. Analizar cada grupo de producciones
22     for primer_token, lista_tokens in grupos.items():
23         # Si un grupo tiene más de 1 producción, hay conflicto
24         # (prefijo común)
25         if len(lista_tokens) > 1:
26             # Encontrar la longitud del prefijo común
27             min_len = min(len(t) for t in lista_tokens)
28             prefijo = []
29
30             for i in range(min_len):
31                 token_actual = lista_tokens[0][i]
32                 if all(t[i] == token_actual for t in
33                     lista_tokens):
34                     prefijo.append(token_actual)
35                 else:
36                     break
37
38             # Guardar la nueva regla factorizada (Prefijo +
39             # Nuevo_NT)
40             str_prefijo = " ".join(prefijo)
41             nuevas_reglas_base.append(f"{str_prefijo}
42                                     {nuevo_nt}")
```

```

    ))
49
50     # 3. Ensamblar el diccionario final
51     resultado = {no_terminal: nuevas_reglas_base}
52     if nuevas_reglas_prima:
53         resultado[nuevo_nt] = nuevas_reglas_prima
54
55     return resultado
56
57     # =====
58     # EJECUCIÓN DEL CASO PRÁCTICO PARA EL INFORME
59     # =====
60     if __name__ == "__main__":
61         # Caso 2.3.1 c) Patologías de las gramáticas
62
63         # Ejecutamos la función
64         gramatica_optimizada = factorizar_izquierda(estado_inicial,
65                                                     reglas)
66
67         print("\n=== GRAMÁTICA RESULTANTE OPTIMIZADA ===")
68         for nt, prods in gramatica_optimizada.items():
69             # Unimos las reglas con el símbolo '|'
70             reglas_str = " | ".join(prods)
71             print(f"{nt} -> {reglas_str}")

```

Salida en Python:

```

=== GRAMÁTICA ORIGINAL (CON AMBIGÜEDAD DE PREFIJO) ===
S -> if E then S
S -> if E then S else S
S -> a
S -> b

=== GRAMÁTICA RESULTANTE OPTIMIZADA ===
S -> if E then S S' | a | b
S' -> ε | else S

=== Code Execution Successful ===

```

(Caso Práctico: Ajedrez)

Notación PGN Simplificada

La notación PGN estándar registra partidas completas de ajedrez incluyendo metadatos (jugadores, fecha, resultado) y movimientos detallados. Para efectos de este desarrollo analítico y computacional, nos enfocaremos exclusivamente en el reconocimiento de una secuencia elemental de jugadas (un movimiento de blancas seguido de un movimiento de negras), el cual representa el motor sintáctico básico del juego.

Definición del Subconjunto Simplificado de PGN

Definimos formalmente nuestro lenguaje mediante los siguientes componentes algebraicos: Alfabeto (Σ): Compuesto por los caracteres válidos para representar piezas, casillas, números de jugada y espaciados.

$\Sigma = \{ K, Q, R, B, N, a, b, c, d, e, f, g, h, 1, 2, 3, 4, 5, 6, 7, 8, ., \text{espacio} \}$

Sintaxis de una Jugada Estándar:

- **Indicador de turno:** Un número entero positivo seguido de un punto (ej. 1., 2., 12.).
- **Movimiento de Blancas:** Inicial de la pieza en mayúscula (opcional, ya que los peones no llevan inicial) seguida de la casilla de destino (una letra de la a a la h y un número del 1 al 8).
- **Espacio en blanco.Movimiento de Negras:** Misma estructura que el movimiento de blancas (Pieza opcional + Casilla).

Ejemplos de cadenas válidas en este subconjunto:

- **1.e4 e5** (Movimiento clásico de peones)
- **2.Nf3 d6** (Caballo blanco a f3, peón negro a d6)
- **12.Qd1 Qh5** (Dama blanca a d1, dama negra a h5)

Diseño de la Expresión Regular (Regex)

Para construir la expresión regular de manera limpia y legible, definiremos primero tres macro-componentes sintácticos:

Número de jugada (N): Uno o más dígitos seguidos de un punto.

$$N = [1 - 9][0 - 9]^* \backslash$$

Pieza (P): Iniciales válidas de las piezas en mayúscula o vacío (para el peón).

$$P = [KQRBN]^?$$

Casilla (C): Una letra de la 'a' a la 'h' seguida de un número del '1' al '8'.

$$C = [a - h][1 - 8]$$

Combinando estos componentes para formar la estructura formal de una jugada completa (Número + Blanco + Espacio + Negro), obtenemos la Expresión Regular Global:

$$Regex = [1 - 9][0 - 9]^* \backslash . [KQRBN]^? [a - h][1 - 8] [KQRBN]^? [a - h][1 - 8]$$

Diseño del Autómata Finito Determinístico (AFD)

A partir de la Regex expresada, modelamos un autómata matemático de memoria limitada para el reconocimiento de patrones válidos.

Definición Formal del AFD

El autómata se define como una $5 - M = (Q, \Sigma, \delta, q_0, F)$ donde:

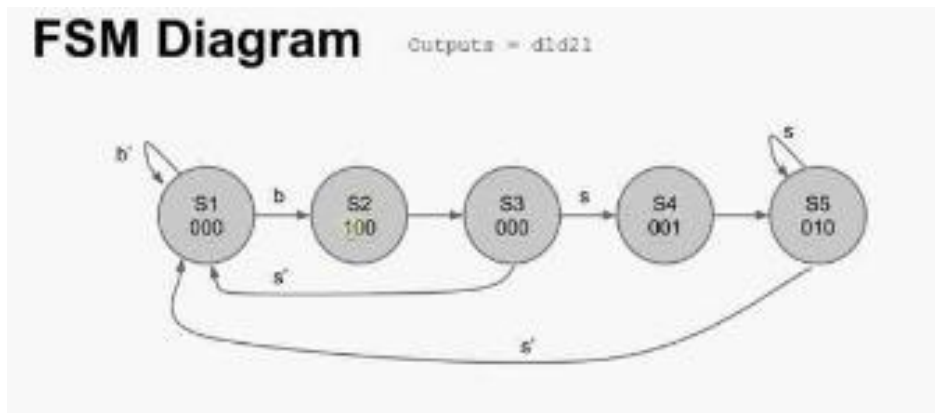
- **Q (Conjunto de Estados):** (q0, q1, q2, q3, q4, q5, q6, q7, q8, q9, q10)
- **Σ (Alfabeto):** El alfabeto de símbolos.
- **q0:** Estado inicial.
- **F (Conjunto de Estados de Aceptación):** q10 Se alcanza al procesar con éxito la jugada de las negras.
- **δ (Función de Transición):** Definida detalladamente en la siguiente tabla.

Tabla de Transiciones del AFD

Cualquier transición no especificada en la tabla se asume que redirige a un Estado de Error (qe), provocando el rechazo inmediato de la cadena por parte del analizador léxico/sintáctico.

Estado Actual	Símbolo de Entrada	Siguiente Estado	Descripción del Paso Técnico
q0	1-sept	q1	Lee el primer dígito del número de jugada.
q1	0-9	q1	Permite números de jugada de múltiples dígitos (ej. 10.).
q1		q2	Reconoce el punto del indicador de turno.
q2	KQRBN	q3	Blancas: Se mueve una pieza mayor (Rey, Dama, Torre, Alfil, Caballo).
q2	a-h	q4	Blancas: Se mueve un Peón (va directo a la columna).
q3	a-h	q4	Blancas: Lee la columna de destino tras la pieza.
q4	1-ago	q5	Blancas: Lee la fila de destino. ¡Movimiento blanco completado con éxito!
q5	espacio	q6	Lee el espacio obligatorio de separación entre jugadas.
q6	KQRBN	q7	Negras: Se mueve una pieza mayor.
q6	a-h	q8	Negras: Se mueve un Peón.
q7	a-h	q8	Negras: Lee la columna de destino tras la pieza.
q8	1-ago	q10	Negras: Lee la fila de destino. Estado Final de Aceptación.

Diagrama del Autómata Finito Determinista (AFD)



Código fuente del analizador PGN

```
1  import sys
2
3  def analizador_lexico_pgn(cadena):
4      # Definición de conjuntos de símbolos del alfabeto
5      piezas = {'K', 'Q', 'R', 'B', 'N'}
6      columnas = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'}
7      filas = {'1', '2', '3', '4', '5', '6', '7', '8'}
8      digitos_no_cero = {'1', '2', '3', '4', '5', '6', '7', '8', '9'}
9      todos_los_digitos = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'}
10
11     # Estado inicial
12     estado = "q0"
13
14     print(f"Analizando cadena: '{cadena}'")
15     print(f"[{estado}] -> Inicio del análisis")
16
```

Se definen los conjuntos de símbolos del alfabeto Sigma, desde las piezas rey, alfil, caballo, torre, reina y peon, las filas y columnas del tablero y los números posibles para los estados de las jugadas

```

for i, caracter in enumerate(cadena):
    estado_anterior = estado

    #Transiciones del AFD
    if estado == "q0":
        if caracter in digitos_no_cer:
            estado = "q1"
        else:
            estado = "qe"

    elif estado == "q1":
        if caracter in todos_los_digi:
            estado = "q1"
        elif caracter == ".":
            estado = "q2"
        else:
            estado = "qe"

    elif estado == "q2":
        if caracter in piezas:
            estado = "q3"
        elif caracter in columnas:
            estado = "q4"
        else:
            estado = "qe"

    elif estado == "q3":
        if caracter in columnas:
            estado = "q4"
        else:
            estado = "qe"

    elif estado == "q4":
        if caracter in filas:
            estado = "q5"
        else:
            estado = "qe"

```

Este es el bucle principal donde se validan cada una de las expresiones caracter por caracter por medio de "enumerate" que permite leer la cadena de caracteres en la posición en la que se suministra, de tal manera que al compilador se le hace cómodo verificar cada caracter.

Validando así que se cumplan las reglas sintácticas y semánticas del PGN

```

elif estado == "q5":
    if caracter == " ":
        estado = "q6"
    else:
        estado = "qe"

elif estado == "q6":
    if caracter in piezas:
        estado = "q7"
    elif caracter in columnas:
        estado = "q8"
    else:
        estado = "qe"

elif estado == "q7":
    if caracter in columnas:
        estado = "q8"
    else:
        estado = "qe"

elif estado == "q8":
    if caracter in filas:
        estado = "q10"
    else:
        estado = "qe"

```

Se añade qe, lo que ayuda a señalar los errores léxicos o semánticos en caso de

que existan, de tal manera que se señala donde está el error para ser corregido

En cambio si la cadena es aceptada el AFD la procesa y da la verificación del programa

```

elif estado == "q10":
    # Si ya estamos en el estado de aceptación y vienen más caracteres,
    # la cadena excede el formato de una jugada simple.
    estado = "qe"

# Si cae en estado de error, rompemos el ciclo (Early exit)
if estado == "qe":
    print(f" Error en carácter '{character}' (posición {i}): Transición inválida desde {estado_anterior}.")
    return False, f"Rechazada: Error sintáctico en la posición {i} ('{character}')"

print(f" Tránsito: '{character}' -> [{estado}]")

# --- Verificación del Estado Final ---
if estado == "q10":
    return True, "¡Cadena ACEPTADA! Sintaxis de jugada PGN válida."
else:
    return False, f"Rechazada: La cadena terminó en el estado incompleto [{estado}]."

```

Se hacen pruebas que validan la cadena a analizar para verificar si el AFD cumple las reglas del PGN en una partida de ajedrez.

```

=== EJECUCION DEL COMPILADOR (ANALIZADOR PGN) ===

Analizando cadena: '1.e4 e5'
[q0] -> Inicio del analisis
Tránsito: '1' -> [q1]
Tránsito: '.' -> [q2]
Tránsito: 'e' -> [q4]
Tránsito: '4' -> [q5]
Tránsito: ' ' -> [q6]
Tránsito: 'e' -> [q8]
Tránsito: '5' -> [q10]
Resultado final: ¡Cadena ACEPTADA! Sintaxis de jugada PGN válida.
-----
Analizando cadena: '12.Nf3 d6'
[q0] -> Inicio del analisis
Tránsito: '1' -> [q1]
Tránsito: '2' -> [q1]
Tránsito: '.' -> [q2]
Tránsito: 'N' -> [q3]
Tránsito: 'f' -> [q4]
Tránsito: '3' -> [q5]
Tránsito: ' ' -> [q6]
Tránsito: 'd' -> [q8]
Tránsito: '6' -> [q10]
Resultado final: ¡Cadena ACEPTADA! Sintaxis de jugada PGN válida.
-----
Analizando cadena: '3.Qd1 Qh5'
[q0] -> Inicio del analisis
Tránsito: '3' -> [q1]
Tránsito: '.' -> [q2]
Tránsito: 'Q' -> [q3]
Tránsito: 'd' -> [q4]
Tránsito: '1' -> [q5]
Tránsito: ' ' -> [q6]
Tránsito: 'Q' -> [q7]
Tránsito: 'h' -> [q8]
Tránsito: '5' -> [q10]
Resultado final: ¡Cadena ACEPTADA! Sintaxis de jugada PGN válida.

```

Sin embargo, se demuestra la el correcto manejo de errores dentro del AFD al suministrar cadenas invalidas, a fin de, someter el AFD a pruebas de estres.

```

Analizando cadena: '1.e9 e5'
[q0] -> Inicio del analisis
  Tránsito: '1' -> [q1]
  Tránsito: '.' -> [q2]
  Tránsito: 'e' -> [q4]
  Error en carácter '9' (posición 3): Transición inválida desde q4.
Resultado final: Rechazada: Error sintáctico en la posición 3 ('9')
-----
Analizando cadena: '1.x4 e5'
[q0] -> Inicio del analisis
  Tránsito: '1' -> [q1]
  Tránsito: '.' -> [q2]
  Error en carácter 'x' (posición 2): Transición inválida desde q2.
Resultado final: Rechazada: Error sintáctico en la posición 2 ('x')
-----
Analizando cadena: '1.e4'
[q0] -> Inicio del analisis
  Tránsito: '1' -> [q1]
  Tránsito: '.' -> [q2]
  Tránsito: 'e' -> [q4]
  Tránsito: '4' -> [q5]
Resultado final: Rechazada: La cadena terminó en el estado incompleto [q5].
-----
Analizando cadena: '01.e4 e5'
[q0] -> Inicio del analisis
  Error en carácter '0' (posición 0): Transición inválida desde q0.
Resultado final: Rechazada: Error sintáctico en la posición 0 ('0')
-----
PS C:\Users\PC1\OneDrive\Documentos\GitHub\LC-VisionLy\Tema 3\Actividad 2.4 PGN Ajedrez>

```

Se observa que en la cadena “1.e9 e5” el 9 a ser colocado como una fila el AFD lo rechaza ya que las reglas del tablero establece que solo existen 8 filas, por lo que se demuestra que el autómata es capaz de encontrar errores y señalarlos a fin de ser corregidos para así validar la jugada.

Por lo que se demuestra que el AFD con una notación PGN en un juego de ajedrez funciona correctamente.

CONCLUSIÓN

El estudio de los lenguajes y gramáticas formales permite comprender las bases matemáticas y computacionales que hacen posible el diseño y funcionamiento de compiladores, intérpretes y numerosos sistemas de procesamiento de información. La relación entre gramáticas y lenguajes demuestra cómo es posible generar y validar estructuras mediante reglas formales bien definidas, garantizando precisión y consistencia en la representación de la información.

La Jerarquía de Chomsky evidenció que existen distintos niveles de complejidad en los lenguajes formales, cada uno con capacidades específicas de representación y procesamiento. Del mismo modo, el análisis de gramáticas libres de contexto permitió apreciar su importancia en la construcción de estructuras sintácticas utilizadas en los lenguajes de programación modernos.

Por otra parte, la identificación y corrección de problemas como la ambigüedad, la recursividad por la izquierda y los conflictos derivados de prefijos comunes demostró la necesidad de aplicar técnicas de optimización gramatical para garantizar un análisis sintáctico eficiente y libre de errores. Asimismo, el estudio de las expresiones regulares y los autómatas finitos confirmó su papel fundamental en el reconocimiento de patrones y en la implementación de analizadores léxicos.

En conclusión, los conceptos desarrollados en este informe constituyen herramientas esenciales para el procesamiento automático de lenguajes, permitiendo modelar, reconocer y validar información de manera formal. Su aplicación trasciende el ámbito de los compiladores, encontrándose presente en áreas como la validación de datos, la inteligencia artificial, la bioinformática y el procesamiento de texto, lo que demuestra su relevancia dentro de la informática contemporánea.

Referencias

- Formal, C. D. un. (s/f). *Las Gramáticas Formales*. Unican.es. Recuperado el 16 de junio de 2026, de https://ocw.unican.es/pluginfile.php/2451/course/section/2492/1-3_Gramaticas_formales.pdf
- la Complejidad, A. S. (s/f). *La Jerarquía de Chomsky*. Unican.es. Recuperado el 16 de junio de 2026, de https://ocw.unican.es/pluginfile.php/2451/course/section/2492/1-4_Jerarquia_Chomsky.pdf
- Wikipedia contributors. (s/f). *Jerarquía de Chomsky*. Wikipedia, The Free Encyclopedia. Recuperado el 16 de junio de 2026, de https://es.wikipedia.org/w/index.php?title=Jerarqu%C3%ADa_de_Chomsky&oldid=163843023
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson Education.
- Edwards, S. J. (1994). *Portable Game Notation Specification and Implementation Guide*. Steven J. Edwards. Recuperado de <https://www.saremba.de/chessgml/standards/pgn/pgn-complete.htm>